

**lwIP-CE with TLS 1.3
Whitepaper**

**Analysis
Justifications**

by Anthony Cagliano

Contents

1	Foreword	2
2	Stack Adaptations and Extensions	4
2.1	Adapting lwIP to the TI-84+ CE	4
2.2	Stack Architecture Overview	5
2.3	USB CDC-Class Ethernet driver	5
2.4	Membuffer ”driver”	7
3	TLS Architecture and Security Model	9
3.1	Threat Model	9
3.2	HWRNG Design and Entropy Model	10
3.3	Certificate Chain Trust Model	18
3.4	Unit Tests and Profiling	21
3.5	Profiling and Mitigating Platform Security Limitations	23
4	Conclusion	28
5	Citations	29
6	Appendixes	30
A	RNG Statistical Dumps	30
B	Differential Timing Analysis Dumps	35

1 Foreword

The **lwIP-CE** project is an adaptation and integration of the Lightweight IP (lwIP) stack for highly constrained embedded systems, specifically targeting the TI-84+ CE platform and similar resource-limited environments. This work represents a deliberate effort to bring modern networking capabilities—including Ethernet, IPv4/IPv6, and TLS—to hardware never originally designed for full internet participation.

Traditional networking stacks assume megabytes of memory, preemptive multitasking, and operating systems with dynamic resource management. The TI-84+ CE provides none of these conveniences. Instead, it offers a single-threaded execution model, limited RAM, no native networking abstraction, and tight timing constraints on a 15 MHz processor. Yet despite these constraints, the goal of lwIP-CE is not merely “minimal networking,” but correct, standards-aligned, interoperable networking.

This whitepaper documents the architecture, design decisions, trade-offs, and implementation details behind lwIP-CE. It covers the layering model from USB driver integration through Ethernet abstraction and into the lwIP stack, as well as the TLS implementation constraints and cryptographic considerations required on this class of device.

lwIP-CE is not intended to turn a calculator into a server farm. It is predominately a proof of concept and capability, an engineering exploration, and a platform for experimentation in secure embedded networking. It demonstrates that:

- Modern TCP/IP stacks can be responsibly adapted to deeply constrained systems.
- TLS with contemporary cipher suites is feasible within tight memory and performance limits.
- Clean architectural boundaries (USB, Ethernet, lwIP, TLS) allow portability and long-term maintainability even in non-traditional environments.

This document is intended for:

- Embedded systems developers
- Networking and protocol engineers
- Security researchers
- Contributors evaluating or auditing the design
- Advanced calculator developers exploring networked applications

The reader is assumed to have familiarity with TCP/IP fundamentals, embedded systems design, and basic cryptographic concepts.

I am fully aware that many readers, particularly within the cryptography community, will invoke the well-known principle: "Don't roll your own crypto". On platforms where mature, audited cryptographic libraries exist, that principle is not only sound but essential. The TI-84+ CE, however, offers no such luxury. When I began work on CryptX several years ago, it was a proof of concept and a personal learning exercise. Over time, it evolved beyond experimentation. The calculator community responded positively, and I discovered that I genuinely valued both the engineering challenge and the responsibility that came with it.

When the scope expanded from standalone cryptographic primitives to a full TLS 1.3 implementation integrated with lwIP, the standards changed. The margin for error narrowed. The expectations rose. This document reflects that shift. Every design decision, optimization, constraint, and trade-off was examined with care. While no implementation on a platform this heavily constrained can claim perfection, this work represents a deliberate, disciplined effort to pursue correctness, interoperability, and security to the fullest extent the hardware permits.

This project – along with the associated research, sustained study and research – is where I learned most of what I now know about cryptography. In that sense, it achieved its core purpose. Even where the implementation is imperfect, the technical growth has been substantial. Early iterations reflected the exploratory nature of the work; over time, the project matured alongside a deeper understanding of cryptographic rigor. I include that candidly, not as self-criticism, but as context for how much this work changed my engineering standards. At the time of writing, I have a materially stronger grasp of the field, including why "good enough" is not adequate for security work, how to design around logical, mathematical, and statistical rigor, and how to document and own security-relevant tradeoffs explicitly.

2 Stack Adaptations and Extensions

Before examining the architecture and technical details, an important disclaimer is necessary.

This implementation is designed to be RFC-compliant and is informed by FIPS-aligned design principles; however, it is not FIPS-certified, nor has it undergone formal third-party cryptographic audit beyond review within the TI-84+ CE development community. Any security assurances provided by this project should therefore be considered provisional and contextual.

These limitations are not the result of neglect, but of constraint. The TI-84+ CE platform imposes strict trade-offs between computational performance, memory usage, and defensive hardening. Certain implementation decisions reflect careful balancing between theoretical ideal and practical feasibility on a 15 MHz, resource-limited system.

That said, this document is written with full transparency. The architecture, cryptographic constructions, mitigation, and design rationale are laid out in detail. Readers are encouraged to evaluate the implementation on its merits. You may find that the conservatism expressed here exceeds what the engineering decisions ultimately warrant.

2.1 Adapting lwIP to the TI-84+ CE

The initial porting effort was undertaken by Cemetch user *commandblockguy*, who prepared lwIP for the TI-84+ CE environment by isolating and conditionally excluding features incompatible with the calculator's execution model. This included removing OS-dependent functionality, threading assumptions, and unsupported memory abstractions, while preserving protocol correctness.

A more substantial challenge arose from the calculator's atypical integer model. The eZ80 CPU operates in a 24-bit address and data environment, and under the CE toolchain, the fundamental C types differ from conventional 32-bit architectures. In particular:

- `short` is 16 bits
- `int` is 24 bits
- `long` is 32 bits (implemented as multi-register operations)

At the time of the original port, support for integer widths beyond 32 bits was limited, requiring careful handling of any operations that assumed native 32-bit promotion semantics. Arithmetic exceeding 24 bits incurs compiler-generated multi-register sequences, and operations beyond 32 bits required explicit

multiprecision handling.

Subsequent revisions of the toolchain introduced additional fixed-width types (e.g., 48-bit and 64-bit integers), allowing portions of earlier compatibility workarounds to be simplified or reverted. Nevertheless, all lwIP fixed-width types were explicitly audited to ensure that their effective widths and promotion behavior matched protocol expectations.

2.2 Stack Architecture Overview

Before beginning this project we needed to itemize what was available and what was needed. The chart below shows the TCP/IP and OSI layer, and what components are assigned to that layer.

TCP/IP Layer	OSI Layer	Component	Function
1 ↔ 2	1–2	USBDRVCE	Hardware transport interface for USB host communication. Handles device enumeration, end-point transfers, and interrupt-driven USB activity used by Ethernet adapters.
2	2	CDC ECM / NCM Ethernet	Ethernet data link layer provided by USB CDC-class adapters. Responsible for Ethernet frame encapsulation, MAC addressing, and link state notifications.
3 ↔ 4	3–4	lwIP	Core networking stack implementing IP, ARP, ICMP, IPv6, TCP, and UDP. Provides packet routing, protocol handling, and socket-style networking interfaces.
5	5–7	ALTCP, TLS, Application	High-level abstractions exposed to calculator programs. ALTCP provides a transport wrapper allowing optional TLS integration, while TLS provides confidentiality, integrity, and endpoint authentication for application data.

Table 1: Relationship between the lwIP-CE networking architecture, the TCP/IP model, and the OSI model.

2.3 USB CDC-Class Ethernet driver

To provide reliable networking on TI-84+ CE hardware, a USB CDC Ethernet host driver was implemented for ECM (Ethernet Control Model) and NCM (Network Control Model) adapters. These are the

common CDC Ethernet profiles available on commodity USB dongles and provide the broadest practical compatibility.

The implementation is layered on `USBDRVCE` (toolchain USB library by jacobly). `USBDRVCE` owns low-level bus/device lifecycle handling; the CDC layer focuses on class parsing, endpoint wiring, and forwarding frames into lwIP.

Initialization starts from the USB callback path and performs descriptor parsing for hubs and CDC-capable devices. The metadata extracted includes:

- The configuration/interface containing the CDC ECM/NCM subclass (vendor-specific configurations often appear first).
- Bulk RX, bulk TX, and interrupt endpoint descriptors.
- MAC address via descriptor string, or fallback control request.
- For NCM, the NCM functional descriptor and NTB capability fields.

After discovery, the driver applies CDC setup:

- (a) `SetConfiguration` to the CDC-capable configuration.
- (b) `SetInterface` to the selected bulk transfer interface.
- (c) If MAC retrieval fails, generate a valid local MAC and program it via control request.
- (d) For NCM we perform the following additional actions:
 - i. Query NTB parameters.
 - ii. Set NTB input size to a small supported size (typically 2048 bytes) to keep RX bursts manageable on CE.
 - iii. Apply packet filter/multicast controls when supported.
- (e) Bind RX/TX/interrupt callbacks into `eth_device_t` and attach the lwIP `netif`.

Interrupt-endpoint handling is intentionally narrow. For CDC ECM/NCM on this platform, only one notification is operationally relevant:

- **NetworkConnection:** maps directly to `netif` link up/down state through `wValue`.

Other CDC notifications (for example `NetworkSpeedChange`) are either non-actionable for this target or already covered at lower driver layers.

RX ingress is decoupled from callback latency via scheduled bulk reads and a ring-buffer drain timer, so frame ingestion remains bounded under memory pressure.

Operational workflow, condensed:

- (a) USB connect/enable events trigger CDC descriptor discovery and configuration.
- (b) Endpoints and MAC identity are resolved; ECM or NCM callbacks are bound.
- (c) Interrupt callback tracks `NetworkConnection` and updates link status.
- (d) RX scheduling/draining timers move bulk data into lwIP while honoring memory-throttle policy.
- (e) Disconnect/disable transitions force link-down and cleanup/reset of endpoint state.

The driver also defines a fatal-error state entered on non-recoverable endpoint stall. In that state, the device is reset, which:

- unschedules callbacks bound to the affected device;
- clears endpoint halt/stall and related transfer error state; and
- triggers a subsequent `ENABLED` event, restarting the normal device-initialization sequence.

This driver provides robust, platform-appropriate USB Ethernet transport for lwIP on TI-84+ CE hardware. By constraining scope to CDC ECM/NCM, prioritizing deterministic recovery on endpoint failure, and decoupling RX ingestion through scheduled drain/throttle control, the implementation achieves stable link behavior under low memory and timing constraints.

2.4 Membuffer "driver"

In addition to the USB driver, a dedicated `membuffer` allocator layer was implemented to keep memory behavior predictable across lwIP, USB RX/TX staging, and TLS components. On this platform, allocator stability is a correctness property: when memory pressure spikes, transport behavior and security-sensitive state handling must degrade in a controlled way rather than fail unpredictably.

The subsystem supports two initialization modes:

- **Dynamic mode:** wraps toolchain `malloc/free/realloc` with explicit heap accounting and pressure reporting.
- **Static mode:** uses a caller-provided arena with fixed bounds and no external allocator dependency after initialization.

Within either mode, `membuffer` provides three buffer classes tuned to workload shape:

- `MEM_BUFFER_RING`: FIFO streaming buffers (used by asynchronous ingress paths such as USB RX).
- `MEM_BUFFER_POOL`: block-style allocations for frequent alloc/free traffic.

- `MEM_BUFFER_FILE`: slot-based, lazy file I/O allocation.

The dynamic path includes pressure-aware sizing controls (grow/shrink thresholds, step sizing, hold logic, and per-buffer max caps) plus pressure callbacks. Pressure levels are tiered (`NONE` → `MILD` → `HIGH` → `SEVERE` → `CRITICAL`), with optional global pressure signaling to coordinate cross-module throttling.

This is directly coupled to the Ethernet ingress path: RX ring buffers use drain callbacks and timer-driven budgets, while pressure callbacks adjust scheduling/throttle behavior under load. In practice, this allows the stack to keep link handling responsive while avoiding runaway allocation during bursts.

From a security perspective, buffers can be flagged with `BUFFER_SECURE_MODE`, enabling automatic zeroization on pop/free/release paths. This does not replace higher-layer cryptographic hygiene, but it materially reduces residual plaintext/key exposure in normal allocator lifecycles.

3 TLS Architecture and Security Model

This section is not a full TLS implementation guide; it focuses only on platform-specific design choices and any standards deviations that require explicit justification.

3.1 Threat Model

Before discussing specific cryptographic constructions and implementation details, it is important to define the threat model under which the lwIP-CE TLS implementation operates. The TI-84+ CE platform was not designed as a secure computing environment, and the security guarantees of this system must therefore be understood within the constraints of the hardware.

This implementation is designed primarily to defend against remote network adversaries while operating on commodity calculator hardware.

Remote Adversaries The primary threat model assumes a remote attacker capable of observing, intercepting, and modifying network traffic between the calculator and a remote endpoint. This includes:

- Passive network eavesdropping.
- Active man-in-the-middle (MITM) attacks.
- Malicious or compromised TLS servers.
- Malformed or adversarial network inputs intended to trigger parser or protocol failures.

The TLS implementation aims to provide confidentiality, integrity, and endpoint authentication against such adversaries, subject to the correctness of the cryptographic primitives and the trust model described in this document.

Local Adversaries A secondary threat model considers attackers with logical access to the device filesystem or software environment. Such attackers may be able to:

- Replace application binaries or calculator variables (AppVars).
- Modify truststore data or configuration files.
- Inspect non-protected memory contents.

Because the platform provides no secure storage primitives or protected execution environment, the system cannot fully defend against a determined local adversary with the ability to modify device storage or execute arbitrary code.

Physical Adversaries The TI-84+ CE provides no hardware root-of-trust, secure enclave, or tamper-resistant memory. An attacker with physical possession of the device may be able to:

- Extract memory contents through debugging interfaces or external tools.
- Observe power, electromagnetic, or timing side channels during cryptographic operations.
- Modify stored data or firmware outside the normal execution environment.

Protection against physical attacks and advanced side-channel analysis is therefore outside the scope of this implementation.

Security Goals Within the above constraints, the lwIP-CE TLS implementation aims to provide:

- Standards-compliant TLS 1.3 handshakes.
- Protection of application data against passive and active network attackers.
- Robust parsing and rejection of malformed protocol inputs.
- Controlled handling of memory and sensitive state where feasible on the platform.

Out-of-Scope Protections The system does not attempt to provide:

- Hardware-backed key protection.
- Resistance to physical device compromise.
- Protection against malicious firmware or operating system modification.
- Strong resistance to advanced side-channel attacks.

Accordingly, the security properties described in this document should be interpreted as network-level protections operating on top of a fundamentally non-secure hardware platform.

3.2 HWRNG Design and Entropy Model

This section defines the evidence model behind the RNG claims in this document. The objective is to keep distinct:

- design reasoning and implementation intent,
- hardware/electrical mechanism,
- formal lower-bound analysis, and
- empirical calibration from device captures.

Randomness is the foundational security dependency for the TLS stack; if the source is weak, all downstream cryptography is weakened regardless of algorithm choice.

The first technical question, therefore, was whether the TI-84+ CE exposes usable entropy, and if so, how that entropy behaves across hardware revisions. Unlike general-purpose systems, the calculator has very few dynamic entropy sources. Early candidates (such as the `r` register and cycle counters) were rejected as predictable. The investigation then shifted to hardware-level behavior in unmapped SRAM regions, using community discussion and direct measurement to build a source model that could be tested and bounded.

This section documents the RNG design rationale and construction, including the statistical and mathematical basis for key decisions, explicit assumptions, identified failure modes, and bounded security estimates. It is not a claim of NIST SP 800-90* compliance or certification. Its purpose is to show that the design was developed against the NIST model and evaluated deliberately rather than implemented ad hoc.

Entropy Characterization The short answer is yes: the calculator does expose usable entropy, but through a much narrower and weaker physical mechanism than modern general-purpose systems. Cemetch user *Zero* summarized the underlying behavior as follows:

SRAM has a pair of bitlines for each bit of each column address, which are connected to a circuit that tries to make them both be charged to an equal, high level before reading from a memory cell (which partially discharges one of the two) and to a sense amplifier (basically a comparator) to read the resulting state afterward. (There may also be another level of gating between the bitlines and the comparators, but that does not strongly affect the dynamics.) In the unmapped space, there are no memory cells, so the controller equalizes the bitlines however well it does, then does nothing (there being no memory cell to discharge them), then senses which bitline is at a higher level. For some column addresses, this will be heavily biased one way or the other because the pre-charge transistors on the bitlines are too different, while for others, it will be less biased because they are more similar.

Another source of non-randomness distinct from the bias is that the bitlines act like capacitors, so precharging only moves their voltage toward the desired level rather than reaching it. This results in a correlation between reads, with the correlation getting weaker with a longer time interval between them and with more intervening reads performed. This is why we cannot just use a von Neumann extractor (which only de-biases) and instead have to do something like XORing many consecutive bits together (which does not fully remove the bias and correlation but can lower it to an undetectable level).¹

A clearer restatement followed later in SAX (Cemetch’s built-in IRC-style chat room):

¹<https://www.cemetch.net/forum/viewtopic.php?p=293079>
Producing crypto-safe randomness on the TI-84+ CE.

The circuit involved [in generating entropy] is an SRAM sense amplifier connected to bit lines with no SRAM memory cells at the relevant addresses. So [the circuit] tries to equalize the voltage on the lines (but not quite due to thermal & noise fluctuations) & then uses a comparator to decide which is higher (also subject to noise near the decision boundary).

For readers with a computer engineering background, these mechanics are expected. The key constraint is that, unlike desktop or server platforms that combine many entropy inputs, this platform relies on a single dominant source region with two defining properties:

- Entropy introduced through noise
- Bias and correlation introduced via lazy voltage balancing behavior by the controller

Designing around this required an iterative approach. At the outset, my understanding of entropy modeling and conditioning was limited; through repeated testing, literature review, and community technical feedback, the design converged into a defensible source-selection and conditioning model.

Entropy Tap Selection Model The generator selects an entropy tap via the following algorithm:

- Set monobit-min (current best bit) to range $[512, 1536]$, to effect a worst-case bias of $\max(p, 1-p)$, $p \in [0.25, 0.75]$.
- Iterate over each bit in the SRAM region defined above, performing a monobit test over 1024 reads for each bit sampled.
- Reject any bit whose monobit count does not outperform the current best bit.
- If the bit being tested outperforms the current best bit, it becomes the current best bit.
- Once the entire region is evaluated, the tap is selected.

The following **derived assumptions** apply:

- $p \in [0.25, 0.75]$ for the probability of a 1 on the selected raw bit is guaranteed at init/re-init, notwithstanding sudden entropy collapse between health checks. The generator fails outright if no qualifying bit is identified.
- Tap selection does not early-exit after finding an acceptable tap; it evaluates the full candidate set, which makes runtime deterministic with respect to scan coverage.

Entropy Tapping Model The generator returns a pseudorandom `uint64_t` via the following algorithm:

- Read from the entropy tap into a 119-byte entropy pool.

- XOR multiple reads, n , into a single composite byte per byte of the entropy pool.
- n began as a data-justified heuristic estimate by Zeroko, k was later computed over observable datasets (see Section 3.2, “Calibrating for Correlation”).
- Hash the entropy pool to soften entropy distribution.
- Compress digest in blocks to output size.

The following **derived assumptions** apply:

- Entropy is credited only to the selected source bit; all other bits in each raw byte are treated as zero-entropy in the bound.
- XOR conditioning is modeled with effective samples, not independence: $n_{\text{eff}} = \lfloor n/k \rfloor$.
- Conditioning and hashing do not create entropy; they only redistribute/extract available source entropy.
- Per-byte min-entropy after conditioning is bounded by $H_{\infty}(p_{\text{out}})$, with p_{out} derived from the calibrated p_{in} , n , and k .
- Pool-level entropy is bounded as $m_{\text{eff}} \cdot H_{\infty}$, where m_{eff} uses the same correlation discount model.
- Runtime health checks are assumed to detect gross source failure, but short-window entropy collapse between checks remains a residual risk.

Calibrating for Correlation

This subsection estimates the correlation factor k and the resulting $n_{\text{eff,byte}}$ used by the effective-sample model. We collected 1,179,648 bytes (≈ 1.2 MB) of source output per device, split evenly across two conditioning modes: no whitening (R0) and 16 XORs (R17). The capture size, though small relative to what some might ask for, was intentionally capped for platform constraints:

- Total calculator storage (RAM + Archive) is roughly 2 MB.
- Archive space is partially consumed by resident applications and system artifacts.
- SRAM tap behavior is time-varying; long captures increase risk of tap drift during acquisition; continuing collection over re-runs risks choosing different taps, which defeats the purpose.
- Pushing capture size substantially higher increases the chance of garbage collection, out-of-memory faults, and collection instability.

Accordingly, 1.2 MB was selected as a conservative operating point that balances statistical utility against collection reliability. The resulting sample size still provides over 9 million bit observations per mode, sufficient for coarse bias and first-order correlation estimation.

Each mode (R0, R17) was analyzed on computer as an $8 \times N$ bit matrix, with each bit column treated as an independent Bernoulli stream. A targeted subset of Dieharder tests was used for coarse non-uniformity screening and repeated across runs for reproducibility checks. Bit-columns were evaluated independently and never concatenated, since mixing columns would invalidate the per-source-bit interpretation. Full local run output is slated for Appendix A and can be extended with community submissions.

Using one best-bit trace per test run (7 shown of 7 total) from the dataset:

Run	Bit	p_{R0}^*	p_{R17}^*	$ p_{R17}^* - 0.5 $
test1	5	0.8867	0.5126	0.0126
test2	5	0.8935	0.5090	0.0090
test3	5	0.5369	0.5003	0.0003
test4	2	0.5861	0.5003	0.0003
test5	5	0.8852	0.5073	0.0073
test6	3	0.8458	0.5005	0.0005
test7	3	0.9608	0.6057	0.1057

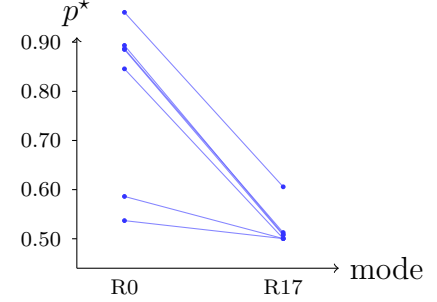


Figure 1: Per-test best-bit progression from raw source (R0) to conditioned output (R17), plotted as $p^* = \max(p, 1 - p)$ so bias direction is folded symmetrically. Each polyline is one test trace from the table above. Full dataset will be available in Appendix 1.

On this subset, mean absolute bias contracts from 0.2993 (R0) to 0.0194 (R17), consistent with strong whitening under 17-read conditioning.

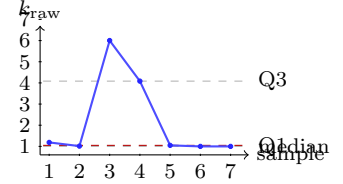
This extract, together with the larger dataset, is consistent with the generator constraint: source selection (R0) requires at least one bit no worse than 75/25 bias. Accordingly, the conservative proof domain is set to $p \in [0.5007, 0.7500]$, with bounds evaluated through $\max(p, 1 - p)$ to cover both bias directions symmetrically.

$$k_{\text{eff}}(n) = \min\left(17, \max\left(1, \frac{n \ln|1 - 2p_{\text{in}}|}{\ln|1 - 2p_{\text{out}}|}\right)\right), \quad n = 17$$

For implementation-level bounds in this section, k is hard-capped at 17 (the total reads in the 1-read + 16-XOR path) to avoid divergence in the inversion as $p \rightarrow 0.5$.

Across all $7(p_{\text{in}}, p_{\text{out}})$ pairs from the dataset (using R0 as p_{in} and R17 as p_{out}), the raw inversion values are:

Run-bit	p_{in}	p_{out}	k_{raw}	Metric	Value
test1-bit5	0.8867	0.4874	1.188	Median k	1.049
test2-bit5	0.8935	0.4910	1.014	Mean k	2.191
test3-bit5	0.5369	0.4997	6.002	σ (pop.)	1.876
test4-bit2	0.5861	0.4997	4.084	Q1	1.000
test5-bit5	0.8852	0.4927	1.049	Q3	4.084
test6-bit3	0.8458	0.4995	1.000	IQR	3.084
test7-bit3	0.0392	0.3943	1.000	Defensible range	[1.000, 6.002]



From these traces, we take $k \in [1.000, 6.002]$ as a defensible empirical range and carry forward $k_{\text{median}} = 1.049$ as the central working value.

For this proof section, the conservative accounting model is:

- only one entropy-bearing bit is credited per pool byte, and
- each credited bit is conditioned by 17reads (1 read + 16 XOR reads).

With correlation discounting, this gives:

Per-bit (conditioning depth)

$$n_{\text{eff,byte}} = \left\lfloor \frac{17}{k} \right\rfloor$$

For $k = 1.049$:

Per-bit result

$$n_{\text{eff,byte}} = \left\lfloor \frac{17}{1.049} \right\rfloor = 16$$

Per-output (u64)

$$n_{\text{eff,pool}} = 119 \cdot n_{\text{eff,byte}} = \left\lfloor \frac{17 \cdot 119}{k} \right\rfloor$$

Per-output (u64)

$$n_{\text{eff,pool}} = \left\lfloor \frac{17 \cdot 119}{1.049} \right\rfloor = 1928$$

Min-Entropy We now combine the median correlation coefficient with the worst-case bias assumption $p = 0.7500$ to compute a minimum effective entropy bound.

Using n_{eff} , compute the conditioned Bernoulli parameter per-bit and per-output:

Per-bit conditioning

$$p_{\oplus} = \frac{1 - (1 - 2p)^{n_{\text{eff}, \text{byte}}}}{2}$$

$$p_{\oplus} = \frac{1 - (1 - 2(0.7500))^{16}}{2}$$

$$p_{\oplus} \approx 0.49999$$

Per output (u64)

$$p_{\oplus} = \frac{1 - (1 - 2p)^{n_{\text{eff}, \text{pool}}}}{2}$$

$$p_{\oplus} = \frac{1 - (1 - 2(0.7500))^{1928}}{2}$$

$$p_{\oplus} \approx 0.50000$$

Now that we have fixed $p_{\oplus}$, we can compute min-entropy over a single bit and over the entire output.

Per-bit computation

$$H_{\infty}(p) = -\log_2(\max(p, 1 - p))$$

$$H_{\infty}(p) = -\log_2(0.49999)$$

$$H_{\infty}(p) \approx 0.99998$$

Per output (u64)

$$H_{\infty}(p) = -\log_2(\max(p, 1 - p))$$

$$H_{\infty}(p) = -\log_2(0.50000)$$

$$H_{\infty}(p) \approx 1.00000$$

As modeled above, the conditioned output trends to approximately one bit of min-entropy per output bit under the stated assumptions. This satisfies the expected entropy density for secure random artifacts.

The generator's primitive output width is `u64`; therefore, larger random artifacts must be produced as sequential blocks, with entropy accounting applied per block and cumulatively.

This is an ultra-conservative reading of the generator, and normal operation is expected to outperform these selected bounds for three reasons:

- The chosen p interval is a worst-case admissible bound. In observed runs, source selection consistently finds at least one bit with lower bias than that bound.
- The selected k is the median of empirical estimates; those estimates include upward excursions driven by inversion instability as p approaches 0.5 (i.e., as $\lim_{p \rightarrow 0.5} \ln |1 - 2p| \rightarrow -\infty$ in the denominator-sensitive regime).
- The proof assigns entropy to only one live bit per source byte. In practice, additional bits may contribute entropy and are also subjected to the same XOR conditioning.

Given this conservative model and empirical calibration, the construction appears suitable for cryptographic randomness generation on this platform, provided it is implemented with proper block-width scaling, reseed and health-check policy, and fail-safe handling on entropy degradation.

Encapsulation The generator can be invoked via simple means by simply using the `tls_random()` function, or for filling buffers, `tls_random_bytes()`. This allows a user to quickly generate cryptographically

suitable randomness without needing to initialize the entire TLS stack. However, this places responsibility for ensuring the continued robustness of the entropy tap on the user, and a function is exposed for this very purpose: `tls_rng_healthcheck()`.

In this mode, entropy output is not pathed through a DRBG, but is instead the direct result of:

- Reading from the entropy tap to a 119-byte entropy pool, with each byte-read being a composite of 17 XOR operations, totaling to 2023 reads for one gathering step.
- Running SHA256 over the entropy pool.
- XOR compression of the resulting digest into a u64

This mode relies on the assumption that the tap is stable and that since the user owns the call-stack, they are responsible for ensuring this at the time they read entropy.

Things get a bit more hands-off when the TLS stack is operational given that the stack will occasionally request random bytes invisibly. `Tls_init()` enables two timers, one for generator seeding and one for source health checking. Entropy can be provided asynchronously by means of a callback-style *entropy request* (`tls_request_random_bytes()`).

```
bool tls_request_random_bytes(uint8_t *out, size_t len, tls_random_request_cb_t cb, void *arg, bool
↪ blocking);
```

This:

- Registers a request for random bytes, a buffer to return those bytes into, and a callback to run when the request completes.
- If `blocking=true`, execution halts until enough entropy is returned to fill the request.
- If `blocking=false`, execution returns to the caller immediately; the callback function fires when the request is fulfilled.

While being used within TLS, entropy is returned through a SHA-256 DRBG serviced by a 50 ms request timer. On each timer cycle, the RNG path behaves as follows:

- A health-check gate runs first. If source health fails, the gather step aborts and no new DRBG material is accepted for that cycle.
- One `tls_random()` value (8 bytes) is collected.
- **If the DRBG is not yet instantiated:** those 8 bytes are appended to seed material. Once 32 bytes total are accumulated, the DRBG state is instantiated as `SHA256(seed_material)`.

- **If the DRBG is already instantiated:** the new 8-byte sample is mixed by reseeding the state with a SHA-256 update over current state, entropy chunk, and counter.
- An entropy budget is credited by 8 bytes each successful gather, capped at 32 bytes.
- Active requests are fulfilled from DRBG output only when budget is available; output decrements the budget by the number of bytes returned.

In blocking mode, `tls_request_random_bytes()` runs this same gather/fulfill loop synchronously until complete (or retry limit). In non-blocking mode, requests are queued and completed by the timer path.

Additionally, a RNG source health check is also serviced by a 30s timer that runs the following statistical analysis:

- Request 32 random bytes through `tls_random()`. Note that we are not using DRBG-conditioned output.
- If sample matches last sample, fail health check.
- Test monobits (total 1 bits across sample). If outside range $[88, 168]$, fail health check.
- Test max equal byte run. If max run exceeds 4, fail health check.

These tests and sample sizes represent a tradeoff between test quality and the calculator's available computation.

If quick statistical testing demonstrates that the source has degraded beyond acceptable levels, the generator initialization function is called, which should cause a new source to be selected. Given the abstraction between tap, the entropy pool, and the DRBG, choosing a new entropy tap is invisible and does not impact upstream state.

3.3 Certificate Chain Trust Model

A core requirement for Internet security is authenticating that the remote endpoint is actually the party it claims to be. In TLS, that assurance is normally derived from certificate-chain validation against a trust anchor.

On this platform, however, certificate handling introduces three practical constraints:

- Certificate chains can be large (often multiple kilobytes), which is expensive relative to available heap.
- Full-path X.509 processing and repeated signature checks are computationally heavy on the CE.
- As with the RNG capture constraints discussed earlier, the CE does not have sufficient storage budget for a desktop-scale trust store.

We needed a trust model that remained defensible for the intended use case while still allowing the calculator to complete a handshake without experiencing a nuclear meltdown in its CPU. We settled on a signed SPKI-pinning trust model.

In this model, trust is expressed as pinned **SubjectPublicKeyInfo** (SPKI) identities in a signed truststore. During certificate processing, the implementation attempts to locate a matching trusted SPKI and then validates the certificate path consistent with that anchor.

Operationally, the acceptance path is:

- parse and validate the presented certificate chain structure;
- verify chain signatures and certificate validity constraints required by this implementation;
- locate a matching trusted SPKI identity from the signed truststore; and
- accept only if both cryptographic chain validation and truststore identity matching succeed.

This approach is intentionally narrower than a full desktop-class WebPKI verifier. The tradeoff is reduced processing and memory pressure in exchange for an explicit, managed trust boundary defined by the shipped truststore.

Trust Store Generation and Specification

The truststore is generated by a scheduled GitHub Actions workflow (quarterly) and published as a signed TI AppVar (.8xv) for device deployment. Conceptually, it serves the same role as an OS trust-anchor bundle, but in a compact, platform-specific format. The **Store Version** field is a binary schema compatibility contract, not a content revision marker. It is incremented only when serialization/parsing semantics change in a way that affects decoding. Routine truststore content updates, naming changes, and signer-key rotation do not require a version increment.

The workflow:

- (a) Scans common CA-certificate locations on the runner (for example `/etc/ssl/certs`) and collects parseable trust anchors.
- (b) Extracts the following metadata from each certificate:
 - The **SubjectPublicKeyInfo** field
 - The issuer and owner IDs
 - The expiry timestamps (not-before, not-after)
- (c) Serializes a binary truststore with the following layout:

Field	Size	Notes
Signature Field	256 bytes	File signature / authenticity block
Store Version	2 bytes	Truststore schema version
Created Timestamp	4 bytes	Build timestamp
Cert Entry Count	2 bytes	Number of certificate records
Per-certificate record (repeated Cert Entry Count times)		
Owner Id	32 bytes	Truncated if source identifier exceeds 32 bytes
Issuer Id	32 bytes	Truncated if source identifier exceeds 32 bytes
Not-Before	4 bytes	Validity start timestamp
Not-After	4 bytes	Validity end timestamp
SHA-256 SPKI	32 bytes	Digest of SubjectPublicKeyInfo
EOF	–	End of file

- (d) Signs the truststore bytes from `Store Version` through `EOF` using RSA-2048 PSS-SHA-256 and writes the signature into `Signature Field`.
- (e) Stores the signing key in GitHub Secrets, materializes it only in runner temp storage for signing/verification, then deletes it (including `shred` cleanup).
- (f) Converts the binary store to TI AppVar format (`.8xv`) via `convbin`.
- (g) Publishes the resulting AppVar as the `truststore-latest` release artifact.
- (h) Links validation at runtime by shipping the corresponding public verification key within lwIP's truststore module.

Truststore age is currently a soft-fail signal, not an enforced rejection condition. A stale store can trigger warning behavior, but does not by itself block use of an otherwise valid signed truststore. Age checks are meaningful only when time is sane, which requires either (1) a valid local clock state, or (2) a functioning NTP path.

Algorithm agility is version-gated. The current format uses RSA-2048 PSS-SHA-256 for truststore signatures; future signature-scheme migrations are expected to ship behind an explicit format/version transition plus corresponding verifier update.

Certificate Chain Validation

When the TLS module initializes, it also initializes the truststore. Initialization performs:

- (a) Signature verification of the truststore against the embedded public verification key.
- (b) Schema/version compatibility checks against the version expected by lwIP.

If truststore initialization fails, TLS client connections are refused.

When operating as a client and processing the TLS `Certificate` message, the implementation parses the X.509 chain and evaluates each certificate as follows:

- (a) Extract `SubjectPublicKeyInfo` and compute its SHA-256 digest.
- (b) Check whether the digest matches a pinned truststore entry.
- (c) Verify required CA constraints (`CA=true`; if `KeyUsage` is present, `keyCertSign`).
- (d) Optionally (configuration-dependent), enforce validity-window checks and owner/issuer metadata matching.

The connection is accepted only if chain parsing yields at least one certificate that satisfies the trust and constraint checks above. Otherwise, the handshake is terminated.

Compromise and Revocation of Signing Key

If the truststore signing key is suspected or confirmed compromised, the incident response is:

- (a) Revoke and replace the compromised signing key.
- (b) Regenerate the truststore under the replacement key.
- (c) Publish a new lwIP release embedding the replacement public verification key.
- (d) Publish a security advisory across project channels (GitHub, Discord, and community forums).

3.4 Unit Tests and Profiling

This project uses two complementary validation layers: deterministic unit tests and profiling-oriented measurements. Unit tests verify functional correctness, safety checks, and regression resistance for module behavior. Profiling tests characterize timing stability, performance behavior, and platform-dependent variance under controlled workloads.

The unit test suite is structured around explicit acceptance criteria for security-critical paths. Coverage includes truststore integrity verification, certificate-chain trust decisions, RNG initialization/request behavior, parser correctness for ASN.1/X.509/PKCS flows, and expected rejection behavior for malformed or invalid inputs. Tests are built from applicable RFC vectors where available (with RFC and section identifiers recorded in test metadata), and from implementation-directed cases where no official vectors exist. Where policy requires fail-closed behavior, tests assert rejection rather than silent degradation.

Profiling tests are intentionally separated from strict correctness gating. Their role is to quantify behavior, not merely assert it. In particular, timing-oriented tests evaluate differential behavior across input classes

and report whether observed variance remains within calibrated bounds. Because timing consistency evidence is environment-sensitive, profiling outcomes are interpreted as indicators and trend signals rather than formal proofs.

The primary profiling module, `tls_timing`, uses robust per-primitive calibration instead of a fixed global cycle limit. For each primitive, repeated samples are collected on a calibration class (“same input”, currently the best-case class), yielding cycle samples $\{T_1, \dots, T_n\}$. The following robust statistics are then computed:

$$\begin{aligned} m &= \text{median}(T_1, \dots, T_n), \\ \text{MAD} &= \text{median}(|T_i - m|), \\ \sigma_{\text{robust}} &= 1.4826 \cdot \text{MAD}. \end{aligned}$$

For each tuning constant c , the accepted cycle deviation is:

$$\Delta_c = \max(c, c \cdot \sigma_{\text{robust}}).$$

Two c modes are reported for every primitive:

- **Standard mode:** $c = 6$ (used for pass/fail gating in CI).
- **Rigorous mode:** $c = 4$ (reported for additional strictness, not current gate criterion).

Each primitive is then evaluated across input classes (best, worst, random, edge), defined:

- **Best-Case:** Input is filled with `0x00`.
- **Worst-Case:** Input is filled with `0xFF`.
- **Random:** Input is filled with pseudo-randomness.
- **Edge-Case:** First half of the input is filled with `0x00` and the second half is filled with `0xFF`.

For each class, the reported statistics include class median deviation from the calibration median and exceedance rate (percentage of samples where $|T_i - m| > \Delta_c$). A differential failure is flagged when separation from calibration exceeds the accepted deviation with directionally consistent behavior across at least a three-quarters majority of samples.

Each unit is sampled 50 times, where each sample is a composite of repeated executions selected by

algorithm runtime class: fast units use 64 repetitions, medium units use 16, and slow units use 2. This batching further suppresses class-independent timing jitter.

The workflow pipeline runs these layers independently so long-running profiling does not block deterministic unit validation. Unit results provide hard regression gates; profiling results provide ongoing security/performance telemetry and flag areas requiring deeper analysis. Together, they form a practical validation model for a constrained platform: strict correctness checks where determinism is expected, and measured characterization where hardware/runtime noise is unavoidable.

As with the rest of this document, these tests support engineering confidence and change control, but do not constitute external certification. Their purpose is to make assumptions explicit, catch regressions early, and provide reproducible evidence for design and security claims. The timing framework is intended to detect input-dependent regression and structural timing drift, not to serve as a formal leakage proof under adversarial averaging models.

3.5 Profiling and Mitigating Platform Security Limitations

The TI-84+ CE Hardware Security Profile

The TI-84+ CE hardware security profile is sufficiently limited that, while drafting this section, I considered leaving it blank; doing so would not materially change the conclusion.

The TI-84+ CE provides no meaningful hardware root-of-trust, no secure enclave, and no robust anti-tamper boundary. Accordingly, all software security claims in this document are explicitly bounded by that platform reality.

Profile Characteristics

- The platform is built around a Zilog eZ80-class CPU (24-bit addressing mode, 48 MHz class operation), optimized for calculator workloads rather than secure key custody or trusted execution.²
- The platform does not expose a hardware secure enclave, TPM-equivalent, or other dedicated primitive for sealing long-term private key material in user-space workflows.
- Memory and storage interfaces are not designed as tamper-resistant boundaries against a local adversary with physical control of the device and transfer path.
- Security-sensitive software therefore operates as a best-effort hardening layer over commodity calculator hardware, not as a hardware-backed trust anchor.

²<https://ce-programming.github.io/toolchain/static/hardware.html>

- Runtime checks that depend on wall-clock time (for example certificate validity windows or truststore age warnings) are only as reliable as the device clock state or external synchronization path.

From this profile, the implementation **cannot fully defend against**:

- Physical side-channel leakage (power/EM/thermal) from the device during cryptographic operation.
- Local code-execution adversaries that patch binaries, bypass integrity checks, or tamper with persistent containers (for example AppVars).
- Offline extraction or modification of unsealed keying material and trust artifacts when the device/storage path is under attacker control.

From this profile, the implementation **can partially mitigate**:

- Host-assisted memory observation during active cryptographic transforms (for example side-loading plus memory mapping).
- Leakage of secrets or intermediate plaintext $\leftrightarrow$ ciphertext transform state in stack and allocated buffers, through explicit zeroization and guarded handling.

These mechanisms are implemented via the **cryptoguard** wrapper detailed in the next section.

Cryptoguard Wrapper

In an effort to provide modest defense against host-assisted memory observation and leakage of secret-s/plaintext, we implemented a **cryptoguard** module. This module provides wrapper functions around security-sensitive contexts. The module:

- Saves the current interrupt state, then disables system interrupts. This reduces the opportunity for host-assisted observation during sensitive operations, since USB activity handling depends on interrupt-driven processing. On exit, the prior interrupt state is restored.
- Clears stack memory from the current stack location up to the configured stack top before returning to the caller. This reduces residual exposure of stack-allocated buffers and intermediate values after sensitive routines complete.

The current cryptoguard wrapper implementation is included below directly from source:

```

1  include "virtuals.inc"
2
3  ; -----
4  ; C-callable crypto guards (ez80)
5  ; -----
```

```

6   ; tls_crypto_guard_enable() -> save interrupt state, disable interrupts
7   ; tls_crypto_guard_disable() -> clear stack, restore interrupt state
8
9   assume adl=1
10  section .text
11
12  public _tls_crypto_guard_enable
13  public _tls_crypto_guard_disable
14
15  ?stackBot                := 0D1987Eh
16
17  _crypto_guard_state_ptr    db 0,0,0
18  _crypto_guard_state_start  rb 16
19  _crypto_guard_state_end:
20
21  ; void tls_crypto_guard_enable(void)
22  _tls_crypto_guard_enable:
23      ; if pointer is zero check
24      ld hl, (_crypto_guard_state_ptr)
25      ld a, h
26      or l
27      jr nz, .has_ptr
28  ; .no_has_ptr
29      ld hl, _crypto_guard_state_start      ; init pointer to start of state stack
30      jr .save_state
31  .has_ptr:
32      ; if pointer is at end of state stack
33      ld bc, _crypto_guard_state_end
34      or a
35      sbc hl, bc
36      ld hl, (_crypto_guard_state_ptr)
37      ret z          ; do nothing
38  .save_state:
39      ld a, i                      ; interrupt state into a
40      ld (hl), a
41      inc hl
42      ld (_crypto_guard_state_ptr), hl    ; advance state ptr
43      di
44      ret
45
46  ; void tls_crypto_guard_disable(void)
47  _tls_crypto_guard_disable:
48  ; _erase_stack:
49      ; set from stackBot + 4 to ix - 1 to 0
50      scf
51      sbc hl, hl
52      add hl, sp
53      dec hl

```

```

54      ex de, hl
55      ld hl, -(stackBot + 3)
56      add hl, de
57      push hl
58      pop bc
59      scf
60      sbc hl, hl
61      add hl, sp
62      ld (hl), 0
63      lddr
64
65      ; restore interrupts
66      ; do we have the pointer
67      ld hl, (_crypto_guard_state_ptr)
68      ld a, h
69      or l
70      ret z
71      ; is the pointer at the start
72      ld bc, _crypto_guard_state_start
73      or a
74      sbc hl, bc
75      ret z
76      ld hl, (_crypto_guard_state_ptr)
77      dec hl
78      ld (_crypto_guard_state_ptr), hl
79      ld a, (hl)
80      or a
81      ret z
82      ei
83      ret

```

Additional Implemented Security Mechanisms

- **tls_secure_memzero():** A secure zeroization routine intended to avoid compiler elimination of memory-clearing operations. Unlike a plain `memset()`, it uses `volatile`-qualified access so clearing side effects are preserved during optimization.
- **Secure Membuffer Contexts:** As noted in the allocator section, a `membuffer` created with `BUFFER_SECURE_MODE` applies automatic zeroization in key lifecycle paths:
 - when `free()` is called on pool-allocated blocks;
 - when bytes are popped from a ring buffer;
 - in dynamic mode, when the allocation shrinks (the released region is zeroed).

When evaluated in unison, these mechanisms provide best-effort memory sanitization across allocation, use, and teardown paths, reducing residual plaintext/key exposure without claiming complete protection

against a local or physical adversary.

4 Conclusion

Placeholder

5 Citations

Special thanks to the following individuals who contributed materially to the project:

Role	Contributor(s)			
Lead Developer	Anthony Cagliano			
C → eZ80	Adam Beckingham			
Entropy Analysis	Zeroke			
modexp_2048	jacobly			
x25519	Peter Tillema (PT_)			
Other Contributions (Info, Optimizations, Supporting Algorithms)	jacobly	calc84maniac	Zeroke	
	John Caesarz	MateoC		
Testers	Alessio			
Donations				

Source	Use in This Project	Reference
B-Con crypto-algorithms tiny-ECDH-c	Ported reference implementation for AES block/key-schedule routines and SHA-256 core hashing primitives. Reference ECDH implementation basis.	https://github.com/B-Con/crypto-algorithms/ https://github.com/kokke/tiny-ECDH-c
lwIP (upstream)	Upstream TCP/IP stack source lineage used as networking foundation and reference baseline.	https://savannah.nongnu.org/projects/lwip/
RFC 4868	HMAC-SHA-256 construction details and test-vector guidance.	https://datatracker.ietf.org/doc/html/rfc4868
NIST SP 800-38D	AES-GCM mode specification and implementation details.	https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
NIST SP 800-90 Series	RNG design/specification guidance (DRBG and entropy-source validation criteria).	https://csrc.nist.gov/projects/random-bit-generation/sp-800-90
Shemanske, <i>Modern Cryptography and Elliptic Curves</i>	Elliptic-curve background and conceptual reference.	Book

6 Appendixes

A RNG Statistical Dumps

This appendix includes the raw RNG dataset currently present in `datasets/appendix_a` (relative to this `.tex` file at build time).

```
--Test1--
```

```
ingested files: 24
```

```
detected groups: R0, R17
```

```
[R0] files=12 bytes=589824 raw=/Users/acagliano/CEdev/source/lwip-ce/tests/profiling/tls_rng_entropy/t_
↪ ools/reports/rng_capture_R0.bin
```

```
[R0] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit5: p1=0.886705 bias=0.386705 trans=0.200484 mono_z=593.979 runs_z=1.655
```

```
[R0] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R17] files=12 bytes=589824 raw=/Users/acagliano/CEdev/source/lwip-ce/tests/profiling/tls_rng_entropy/t_
↪ tools/reports/rng_capture_R17.bin
```

```
[R17] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R17] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R17] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R17] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R17] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R17] bit5: p1=0.487362 bias=0.012638 trans=0.499055 mono_z=19.411 runs_z=0.961
```

```
[R17] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R17] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
--Test2--
```

```
ingested files: 24
```

```
detected groups: R0, R17
```

```
[R0] files=12 bytes=589824
```

```
[R0] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```
[R0] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

```

[R0] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R0] bit5: p1=0.893451 bias=0.393451 trans=0.189684 mono_z=604.341 runs_z=2.851
[R0] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R0] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] files=12 bytes=589824
[R17] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit5: p1=0.491013 bias=0.008987 trans=0.500186 mono_z=13.805 runs_z=0.535
[R17] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

--Test3--
ingested files: 24
detected groups: R0, R17
[R0] files=12 bytes=589824
[R0] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R0] bit1: p1=0.000176 bias=0.499824 trans=0.000353 mono_z=767.729 runs_z=3.838
[R0] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R0] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R0] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R0] bit5: p1=0.536931 bias=0.036931 trans=0.494896 mono_z=56.727 runs_z=3.669
[R0] bit6: p1=0.005086 bias=0.494914 trans=0.010098 mono_z=760.188 runs_z=1.606
[R0] bit7: p1=0.000002 bias=0.499998 trans=0.000003 mono_z=767.997 runs_z=543.061
[R17] files=12 bytes=589824
[R17] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit1: p1=0.005142 bias=0.494858 trans=0.010240 mono_z=760.102 runs_z=0.788
[R17] bit2: p1=0.000002 bias=0.499998 trans=0.000003 mono_z=767.997 runs_z=543.061
[R17] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit5: p1=0.499688 bias=0.000312 trans=0.499691 mono_z=0.479 runs_z=0.474
[R17] bit6: p1=0.093097 bias=0.406903 trans=0.167932 mono_z=625.003 runs_z=4.217
[R17] bit7: p1=0.000007 bias=0.499993 trans=0.000014 mono_z=767.990 runs_z=102.635

```

```
--Test4--
```

```
ingested files: 24
```


detected groups: R0, R17

[R0] files=12 bytes=589824

[R0] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit2: p1=0.586134 bias=0.086134 trans=0.481665 mono_z=132.302 runs_z=5.534

[R0] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit5: p1=1.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit7: p1=0.565055 bias=0.065055 trans=0.490818 mono_z=99.924 runs_z=1.119

[R17] files=12 bytes=589824

[R17] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit2: p1=0.499669 bias=0.000331 trans=0.499930 mono_z=0.508 runs_z=0.106

[R17] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit5: p1=0.000015 bias=0.499985 trans=0.000031 mono_z=767.977 runs_z=43.916

[R17] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit7: p1=0.500351 bias=0.000351 trans=0.499825 mono_z=0.539 runs_z=0.268

--Test5--

ingested files: 24

detected groups: R0, R17

[R0] files=12 bytes=589824

[R0] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit5: p1=0.885206 bias=0.385206 trans=0.203000 mono_z=591.677 runs_z=0.872

[R0] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] files=12 bytes=589824

[R17] bit0: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit3: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

```
[R17] bit5: p1=0.492698 bias=0.007302 trans=0.500245 mono_z=11.216 runs_z=0.542
[R17] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
[R17] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf
```

--Test6--

ingested files: 24

detected groups: R0, R17

[R0] files=12 bytes=589824

[R0] bit0: p1=0.106038 bias=0.393962 trans=0.187883 mono_z=605.125 runs_z=6.901

[R0] bit1: p1=0.066079 bias=0.433921 trans=0.122691 mono_z=666.503 runs_z=4.559

[R0] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit3: p1=0.845847 bias=0.345847 trans=0.259868 mono_z=531.221 runs_z=2.681

[R0] bit4: p1=0.001855 bias=0.498145 trans=0.003699 mono_z=765.151 runs_z=0.331

[R0] bit5: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] files=12 bytes=589824

[R17] bit0: p1=0.492747 bias=0.007253 trans=0.500048 mono_z=11.141 runs_z=0.237

[R17] bit1: p1=0.460654 bias=0.039346 trans=0.495745 mono_z=60.435 runs_z=1.789

[R17] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit3: p1=0.499464 bias=0.000536 trans=0.499228 mono_z=0.823 runs_z=1.184

[R17] bit4: p1=0.026599 bias=0.473401 trans=0.051649 mono_z=727.143 runs_z=1.970

[R17] bit5: p1=0.000002 bias=0.499998 trans=0.000003 mono_z=767.997 runs_z=543.061

[R17] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

--Test7--

ingested files: 24

detected groups: R0, R17

[R0] files=12 bytes=589824

[R0] bit0: p1=0.990560 bias=0.490560 trans=0.018704 mono_z=753.500 runs_z=0.148

[R0] bit1: p1=1.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit3: p1=0.039225 bias=0.460775 trans=0.075256 mono_z=707.750 runs_z=1.174

[R0] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit5: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R0] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] files=12 bytes=589824

[R17] bit0: p1=0.132863 bias=0.367137 trans=0.225513 mono_z=563.922 runs_z=16.354

[R17] bit1: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit2: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit3: p1=0.394289 bias=0.105711 trans=0.474919 mono_z=162.372 runs_z=4.391

[R17] bit4: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit5: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit6: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

[R17] bit7: p1=0.000000 bias=0.500000 trans=0.000000 mono_z=768.000 runs_z=inf

B Differential Timing Analysis Dumps

The differential timing profiler is a robust timing-assessment tool designed to verify that no input class for a unit exhibits detectable runtime drift beyond a class-independent jitter model. As described in the main text, it estimates σ and derives both standard and strict cycle-deviation thresholds from variance measured in the best-case class.

The tables below report per-unit averaged statistics by model. Each row is shaded green when the unit passes and red when it fails under the profiler criterion (directionally consistent runtime deviation above threshold at least 75% of the time). For release-quality builds, all rows should be green.

Table 2: Timing summary by unit for Rev AA.

Unit	Avg σ	Avg Δ (c6/c4)	MedDev (W/R/E)	Over% c6 (W/R/E)	Over% c4 (W/R/E)
AESCBC-D	2.97	18.00/12.00	0.0/1.0/-1.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESCBC-E	2.97	18.00/12.00	1.0/1.0/1.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESCCM-D	2.97	18.00/12.00	2.0/1.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESCCM-E	5.93	36.00/24.00	1.0/1.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESGCM-DV	2.97	18.00/12.00	1.0/3.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESGCM-E	2.22	13.50/9.00	0.0/-0.5/0.5	0.00%/0.00%/0.00%	0.00%/0.00%/1.00%
BYTES	10.38	62.50/41.50	-1.0/-1.0/-1.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
HKDF	1.48	9.00/6.00	1.5/2.5/1.5	0.00%/5.00%/0.00%	11.00%/16.00%/2.50%
HMAC	1.48	9.00/6.00	0.0/0.0/0.5	0.00%/0.00%/0.00%	0.00%/7.50%/0.00%
PBKDF2	1.48	12.00/8.00	0.5/-0.5/-0.5	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
SHA256	3.71	22.50/15.00	0.5/0.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
X25519	4.45	27.00/18.00	-3.0/-3.0/3.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%

Source files parsed: 2

Table 3: Timing summary by unit for Rev PreA.

Unit	Avg σ	Avg Δ (c6/c4)	MedDev (W/R/E)	Over% c6 (W/R/E)	Over% c4 (W/R/E)
AESCBC-D	1.48	9.00/6.00	0.0/1.0/1.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESCBC-E	1.48	9.00/6.00	0.0/0.0/-1.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESCCM-D	1.48	9.00/6.00	0.0/0.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESCCM-E	4.45	27.00/18.00	1.0/0.0/1.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESGCM-DV	1.48	9.00/6.00	0.0/0.0/-1.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
AESGCM-E	1.48	9.00/6.00	1.0/0.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
BYTES	14.83	89.00/59.00	-5.0/-6.0/-4.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
HKDF	1.48	9.00/6.00	0.0/0.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
HMAC	1.48	9.00/6.00	0.0/0.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
PBKDF2	0.00	6.00/4.00	0.0/0.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
SHA256	4.45	27.00/18.00	1.0/1.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%
X25519	1.48	9.00/6.00	0.0/0.0/0.0	0.00%/0.00%/0.00%	0.00%/0.00%/0.00%

Source files parsed: 1